Contents

Report: ABB → HSiKAN smoke test, `TopKBuilder` refactor, entropy-heuristic plan

Date: 2026-05-10 **Slug:** `abb-hsikan-smoke-and-builder` **Builds on:** `reports/2026-05-10-abb-global-topk.md` (the 25× ABB win) **Plans on the table:** `docs/plans/2026-05-10-entropy-vertex-uniform-cycles/` (4-format, compiled)

Summary

Three small actions executed under auto mode after the ABB landing:

1. **A: PyO3 binding + Python route + 1-seed Epinions smoke test.** Added `enumerate_top_k_cycles_signed_bb_rs` to `hymeko_py`, plumbed `HSIKAN_TOPK_MODE=global_bb` into `signedkan_wip/src/n_tuples.py`. Smoke-tested 1-seed Epinions vs the per-vertex baseline at matched abbreviated config (c3+c4, h=4, 20 epochs).
   - **Result: ABB 2.6× faster end-to-end (57 s vs 149 s) but −6.7 pp AUC (0.575 vs 0.642).** Cause: at K=10 000, all retained cycles are score-1.0 all-negative; the M_e has zero score variance and no per-vertex coverage in the long tail. Verified by trying K=100 000 — AUC dropped further (0.548) as more dilution hit.
   - **Conclusion:** ABB is a 25× win for global top-K consumers but does not transfer as a drop-in for HSiKAN, which needs vertex-uniform cycles.
2. **`TopKBuilder` Strategy/Builder refactor** of the enumeration variants (per CLAUDE.md §7). Wraps the eight `enumerate_top_k_cycles*` free functions behind a fluent terminal-method API; existing call sites untouched, additive only. 3 new tests verify each builder method matches its underlying free function.
3. **Plan-only: entropy-heuristic top-K** at `docs/plans/2026-05-10-entropy-vertex-uniform-cycles/` (4 formats, compiles via `lualatex`). Reframes the original "degree-adaptive m_v" idea around the user's entropy insight: select cycles to maximise the per-vertex incidence-distribution entropy. New `StatefulScorer` trait, `EntropyGainScorer`, `InverseDegreeScorer`, branch-and-bound on entropy gain. Production-promotion gated on a 5-seed paired AUC vs the per-vertex baseline.

All gates green: 49 unit + 9 ABB integration + 3 cycle_cache + 7 friedler tests pass; clippy + fmt clean.

Files touched

Created

File	Purpose
`hymeko_py/src/cycles.rs` (+109 LOC at end)	`enumerate_top_k_cycles_signed_bb_rs` PyO3 binding with `score_kind`/`pruner_kind` matrix dispatch via macros (monomorphised, no dynamic dispatch)

File	Purpose
`hymeko_graph/src/topk_cycles.rs::TopKBuilder` (+106 LOC)	Fluent Builder over global / global_bb / per_vertex variants; thin wrappers around the existing par free functions
`hymeko_graph/src/topk_cycles.rs::tests` (+~85 LOC)	3 builder-vs-free-function parity tests
`docs/plans/2026-05-10-entropy-vertex-uniform-cycles/plan.{tex,pdf,typ,md}`	4-format plan for entropy-heuristic top-K (PDF: 5 pp, 463 kB; uses `\Delta H` for entropy gain notation)

Modified

File	Change
`hymeko_py/src/cycles.rs`	Import `BalanceScorer` / `FractionNegativeScorer` / `LowRootScorer` / `SignProductAbsScorer` / `enumerate_top_k_cycles_par_bb` from `hymeko_graph`
`hymeko_py/src/lib.rs`	Register new `enumerate_top_k_cycles_signed_bb_rs` Python symbol
`hymeko_graph/src/lib.rs`	Re-export `TopKBuilder`
`signedkan_wip/src/n_tuples.py::construct_k`	New `HSIKAN_TOPK_MODE=global_bb` branch routing through the ABB binding

CORE.YAML items touched

None. `hymeko_graph` and `hymeko_py` are not core; `signedkan_wip` is research code (not in CORE.YAML at all). No new dependencies.

Test results

Layer	File	Count	Status
Unit (lib)	`hymeko_graph/src/**/tests`	49	✓ (was 46; +3 builder tests)
Integration (ABB)	`hymeko_graph/tests/abb_global_topk.rs`	9	✓
Integration (CSR sign lookup)	`hymeko_graph/tests/csr_sign_lookup.rs`	3	✓
Integration (Friedler)	`hymeko_graph/tests/friedler_scenarios.rs`	17	✓
`hymeko_py` build	—	—	✓ (maturin develop –release)

Smoke-test measurement

Single-seed Epinions training, abbreviated config (`HSIKAN_MIXED_TUPLES=c3,c4`, `--hidden 4 --n-epochs 20 --seed 0`):

Cycle source	Total wall (load + enum + train)	Test AUC	Macro F1
Per-vertex `m=128` (production-default)	**148.6** s	**0.6416**	0.5518
Global ABB `K=10 000`	**57.0** s (2.6× faster)	0.5755 (−0.066)	0.5338
Global ABB `K=100 000`	34.5 s (4.3× faster)	0.5484 (−0.093)	0.5388

Single-seed only — paper promotion needs the 5-seed paired protocol per memory rule. The smoke test answers the binary "is this useful?" question, which is: not as a per-vertex drop-in. AUC

delta is consistently negative and gets worse with more cycles, confirming the global-top-K cycle source biases toward the highest-score cluster (all-negative balanced 4-cycles, of which there are 1.4M on Epinions) rather than a vertex-uniform sample.

Performance attribution (carried forward from ABB report)

The 25× wall-time improvement for `enumerate_top_k_cycles_par_bb` measured in the prior task remains valid as an algorithm-level result. It surfaces here as the "57 s end-to-end" figure for HSiKAN: of that 57 s, only ~5 s is enumeration; the remaining ~50 s is Python startup + 20 training epochs + JSON output. So the actual enumeration speedup transferred (~110 s → ~5 s on the c4 path) but is masked by the overall pipeline budget.

New / removed dependencies

None. `cargo flamegraph 0.6.12` and `lualatex` (TeXLive) used as developer tools, not added to any manifest. `maturin develop --release` was run to install the updated `hymeko_py` wheel into the active conda env (`hymeko-0.1.0-cp313-cp313-linux_x86_64.whl`).

Open issues / follow-up items

1. **Entropy-heuristic plan ready for implementation approval.** `docs/plans/2026-05-10-entropy-vertex-uniform-cy` is plan-only; implementation gated on user nod.
2. **5-seed paired AUC validation** is required *before* promoting any new cycle source to production-paper-headline. Currently we have:
   - Per-vertex `m=128` 5-seed baseline on Epinions (0.846 ± .0106 from `project_epinions_edge_cr_null_2026_05_10` memory)
   - Global ABB single-seed (0.5755, abbreviated config — not directly comparable)
   - Entropy-heuristic: not measured yet (gated on plan approval)
3. `tools.yaml` still doesn't exist; flagged in two prior reports.
4. **HSiKAN integration of global ABB** is plumbed but unused by default. Opt-in via `HSIKAN_TOPK_MODE=global_bb`. Stays available for any downstream that wants global top-K rather than per-vertex.
5. **`TopKBuilder` is parallel-only.** Sequential dispatch is reachable via `RAYON_NUM_THREADS=1` but not via the builder API. If a `.sequential()` method is wanted later, it's a small addition.

Experiment provenance

Field	Value
Git SHA at task start	`c2d30af08e60de28734432eca5f28b6469bdbb91` (working tree dirty from layered prior tasks)
OS / Kernel	Linux 6.17.0-23-generic x86_64
CPU	AMD Ryzen 7 3700X (16 threads)
RAM	31 GiB
GPU	NVIDIA RTX 2070 SUPER (used during HSiKAN training portion of the smoke test)
Rust	1.92.0 (ded5c06cf 2025-12-08)
Python	3.13 (miniconda3)
`hymeko` wheel	re-installed via `maturin develop --release` after Rust changes
Random seed	0 (single-seed smoke)
Dataset	`signedkan_wip/data/epinions.txt` — sha256 `8120d06a0bb4e65d4b821eba1072647ef3429e4e0a3c02e72bf0c53466`
Workload	`--dataset epinions --hidden 4 --n-epochs 20`, `HSIKAN_MIXED_TUPLES=c3,c4`, `HSIKAN_TOPK_PRUNER=balance`, `HSIKAN_TOPK_SCORER=fraction_negative`
Suppressions	None

Conclusion

ABB is correct and 25× faster on its target workload (global top-K), now exposed through Python via `HSIKAN_TOPK_MODE=global_bb` for any caller that wants it, and through Rust via the new `TopKBuilder::global_bb()`. The smoke test confirms it does **not** drop in as a faster per-vertex replacement for HSiKAN — that needs the entropy-heuristic top-K, plan-drafted at `docs/plans/2026-05-10-entropy-vertex-uniform-cycles/`. Disposition: ship the tools as added; await user approval on the entropy plan before implementing.